

逐函数讲解

结论

理解这份代码时，不要先背每一行，而是先抓住每个函数的输入、输出和责任边界。

app.c

`App_Init()`

作用：初始化所有应用层模块。

调用顺序：

```
Remote_Init();  
Chassis_Init();  
Gimbal_Init();  
Safety_Init();
```

为什么这样写：上电时先让所有模块处于已知状态，避免未初始化变量导致电机异常输出。

`App_Loop()`

作用：主循环调度。

它每次循环都会更新 LED，并按 `ROBOT_CONTROL_PERIOD_MS` 周期调用

`App_ControlTask()`。

`App_ControlTask()`

作用：真正的控制周期。

核心流程：

```
Remote_Update(now_ms);  
Safety_Update(cmd, pitch_target, now_ms);  
Gimbal_Control(cmd, dt_s);  
Chassis_Control(cmd, yaw_feedback, dt_s);
```

为什么云台在底盘前：底盘跟随云台模式需要用到最新 yaw 反馈。

remote.c

`Remote_Init()`

作用：初始化按键状态和遥控命令。

默认进入 `ROBOT_STOP`，这是为了安全。

`Button_Update()`

作用：处理按键消抖、短按、长按、长按连发。

为什么需要它：机械按键会抖动，如果没有消抖，按一次可能触发多次。

`Remote_Update()`

作用：把按键事件翻译成机器人命令。

输出结构体是 `remote_cmd_t`，后续底盘和云台只读这个命令，不直接读按键。

chassis.c

`Chassis_Init()`

作用：初始化底盘状态和四个轮子的 PID。

`Chassis_CalcWheelTarget()`

作用：把 $v_x/v_y/w_z$ 转换成四个轮子的目标速度。

这是底盘控制的运动学核心。

`Chassis_Control()`

作用：执行底盘控制。

流程：

1. 判断是否 STOP。
2. 根据底盘模式修正角速度。
3. 计算四轮目标速度。
4. 读取编码器反馈。
5. PID 计算输出。

6. 限幅后输出给电机。

`Chassis_Stop()`

作用：立即清零底盘输出并复位 PID。

为什么要复位 PID：避免急停期间积分残留，恢复后突然冲一下。

`gimbal.c`

`Gimbal_Init()`

作用：初始化 yaw/pitch 目标角和 PID。

`Gimbal_Control()`

作用：执行云台目标角闭环。

流程：

1. 根据 key2/key3 更新目标角。
2. 对 Pitch 目标角限位。
3. 读取 IMU yaw/pitch 角度。
4. 计算角度误差。
5. PID 输出电机控制量。

`Gimbal_Stop()`

作用：清零云台输出并复位 PID。

`pid.c`

`PID_Init()`

作用：设置 PID 参数、输出限幅和积分限幅。

`PID_Calc()`

作用：计算 PID 输出。

公式：

$$\text{output} = k_p * \text{error} + k_i * \text{integral} + k_d * \text{derivative}$$

`PID_Reset()`

作用：清空积分和上一次误差。

`safety.c`

`Safety_Update()`

作用：每个周期更新安全状态。

`Safety_ShouldStop()`

作用：告诉调度层当前是否必须停机。

`Safety_ReportMotorLimited()`

作用：记录电机输出是否触碰限幅。

`c620_can.c`

`C620_PackCurrentFrame()`

作用：把四路电流命令打包成一帧 8 字节 CAN 数据。

`C620_ParseFeedback()`

作用：解析 C620 电调反馈，得到角度、转速、转矩电流和温度。

`C620_AngleRawToDeg()`

作用：把 0-8191 的原始角度转换成 0-360 度。

Source: E:/ai 产出文件/牛马/竞赛/竞

赛/outputs/robomaster/rm_infantry_control_deep_research_v2/improved_pack/
Core/Src/app.c:56